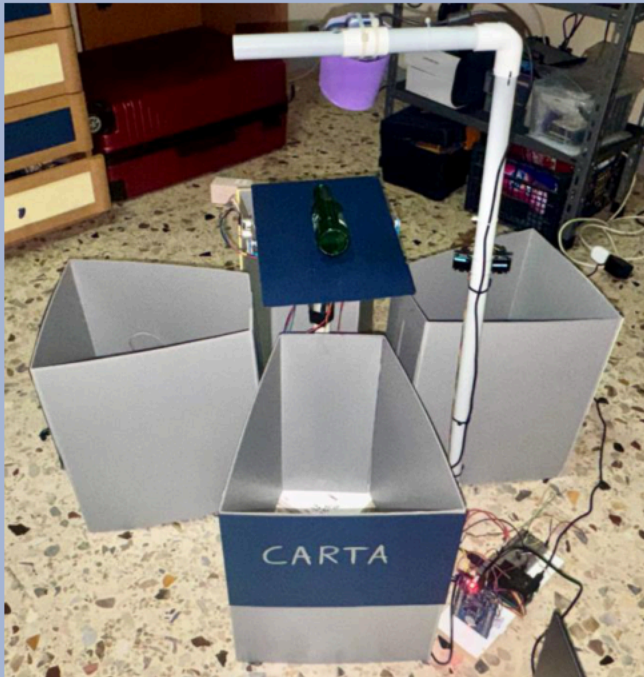


CONCESTINO

SMART WASTE SORTING SYSTEM



Marco Sciaudone DE9000073

Mattia Verrillo DE9000153

Giulio Paesano DE9000080

Laura Verdosci DE9000192

SMART WASTE MANAGEMENT



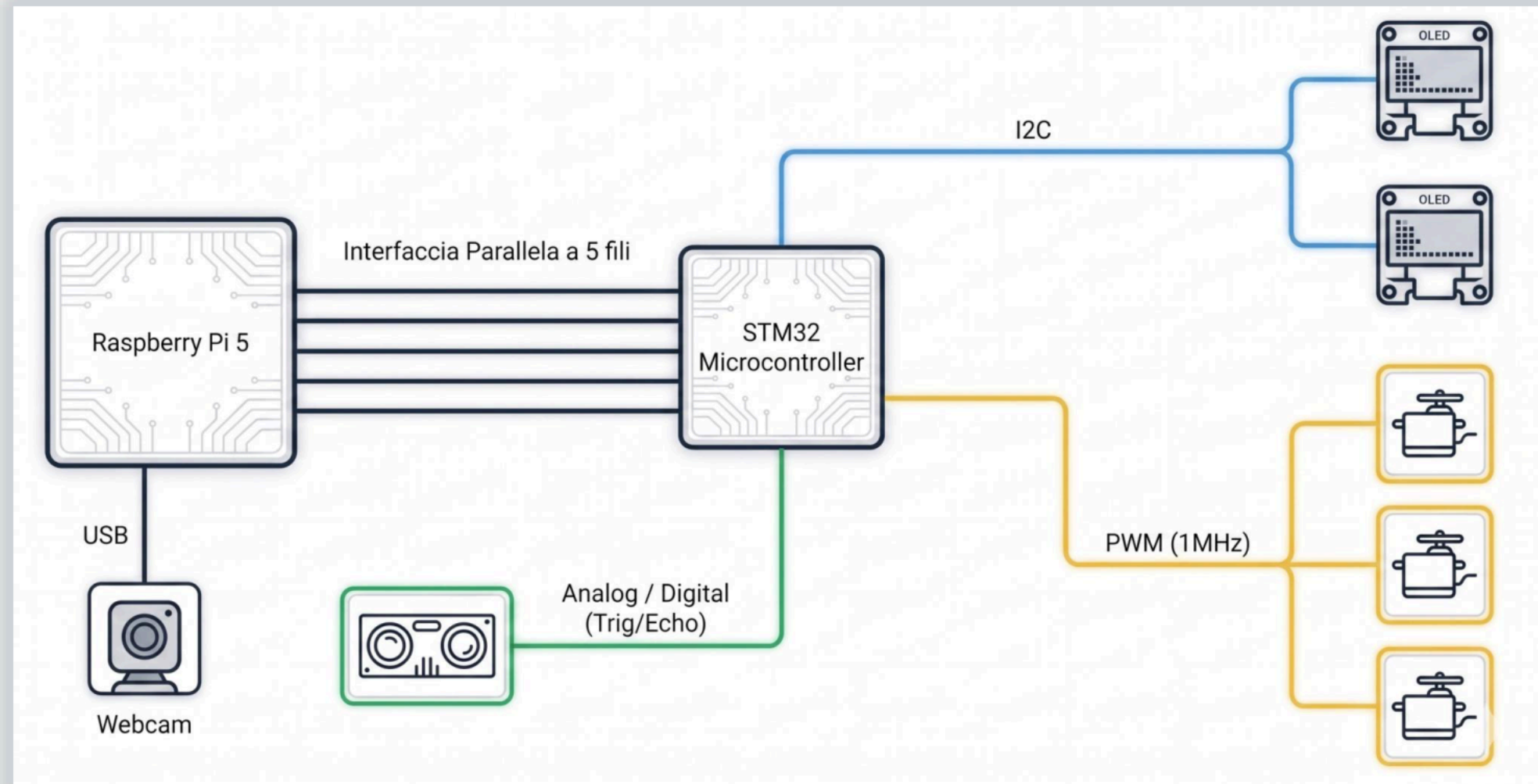
The Problem
Waste classification

depends entirely
on human attention



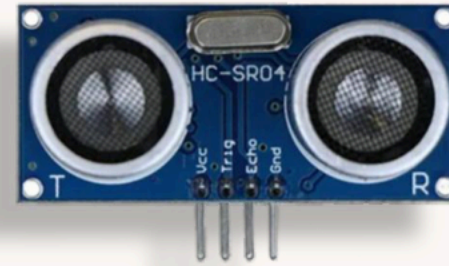
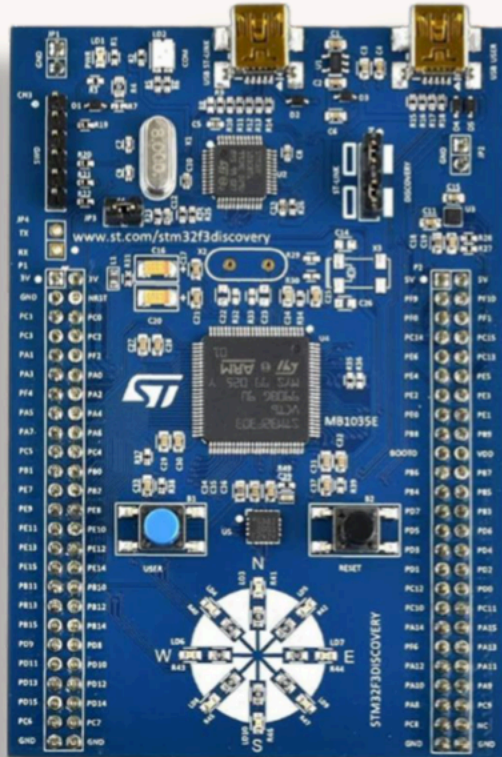
The Solution
Automatic Material
Recognition and Sorting
of Materials

ARCHITECTURE



MAIN HARDWARE COMPONENTS

Raspberry Pi 5
STM32F303DISCOVERY
HC-SR04 Sensor



2 OLED Displays
3 Servomotors



Webcam



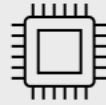
THE ROLE OF THE COMPONENTS



Vision Module

Hardware: Raspberry Pi + Webcam USB

Function: image acquisition and local AI inference for object material recognition



Control Module

Hardware: STM32F303 Discovery

Function: real-time manager of sensors, PWM actuators and communication between peripherals

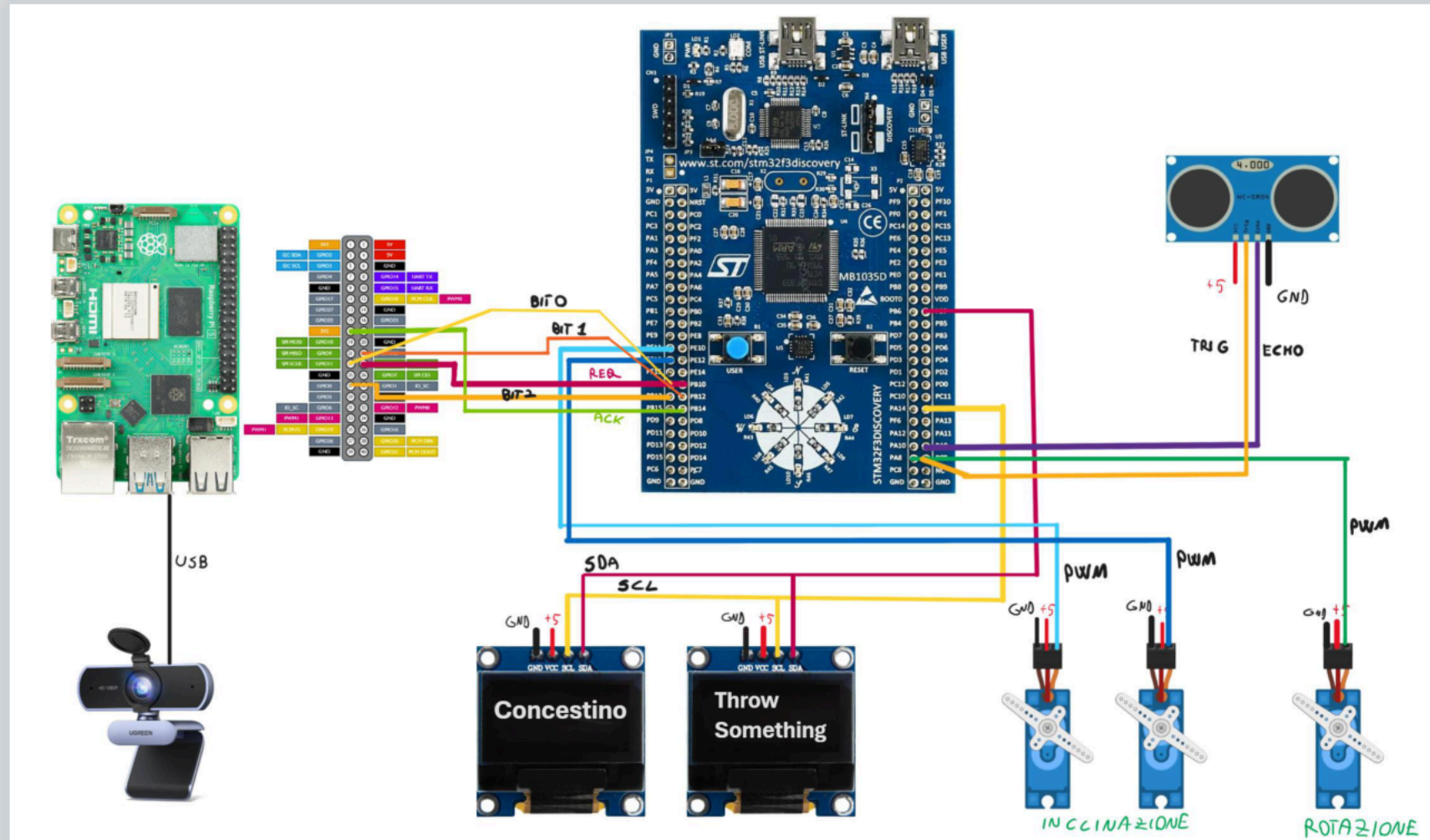


Mechanical Module

Hardware: HC-SR04 + Servo Motors + OLED Display

Function: detection of an object's presence, mechanical disposal action and visual feedback to the user

HARDWARE CONNECTIONS



PROJECT OVERVIEW

Component
Why?

Raspberry Pi 5

1. Sufficient computing power to handle AI model inference

2. USB support for the Webcam

I2C Protocol

Simplicity and optimization for OLED Displays

Parallel Interface

1. Minimum latency

2. Simple interface for the inference result

3. Dedicated STM-Raspberry connection

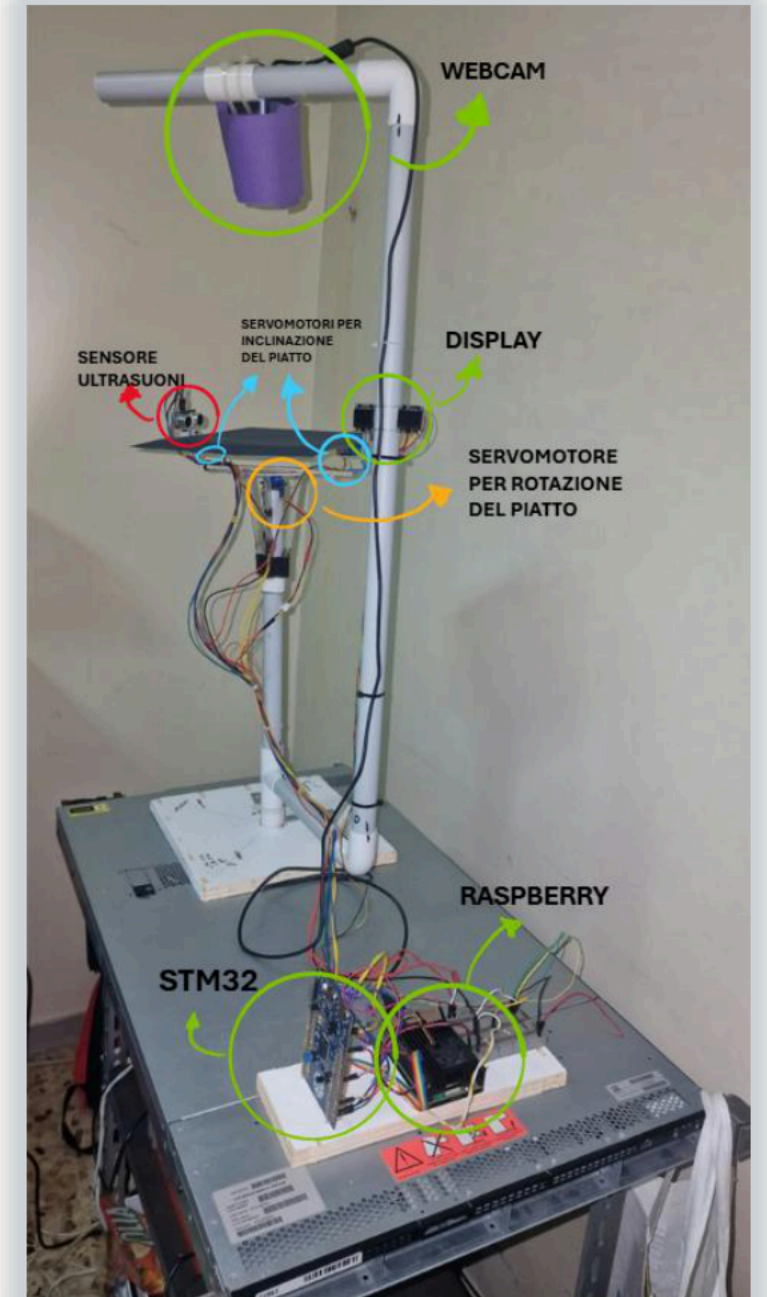
Ultrasonic
Sensor

1. Detects an object regardless of its color

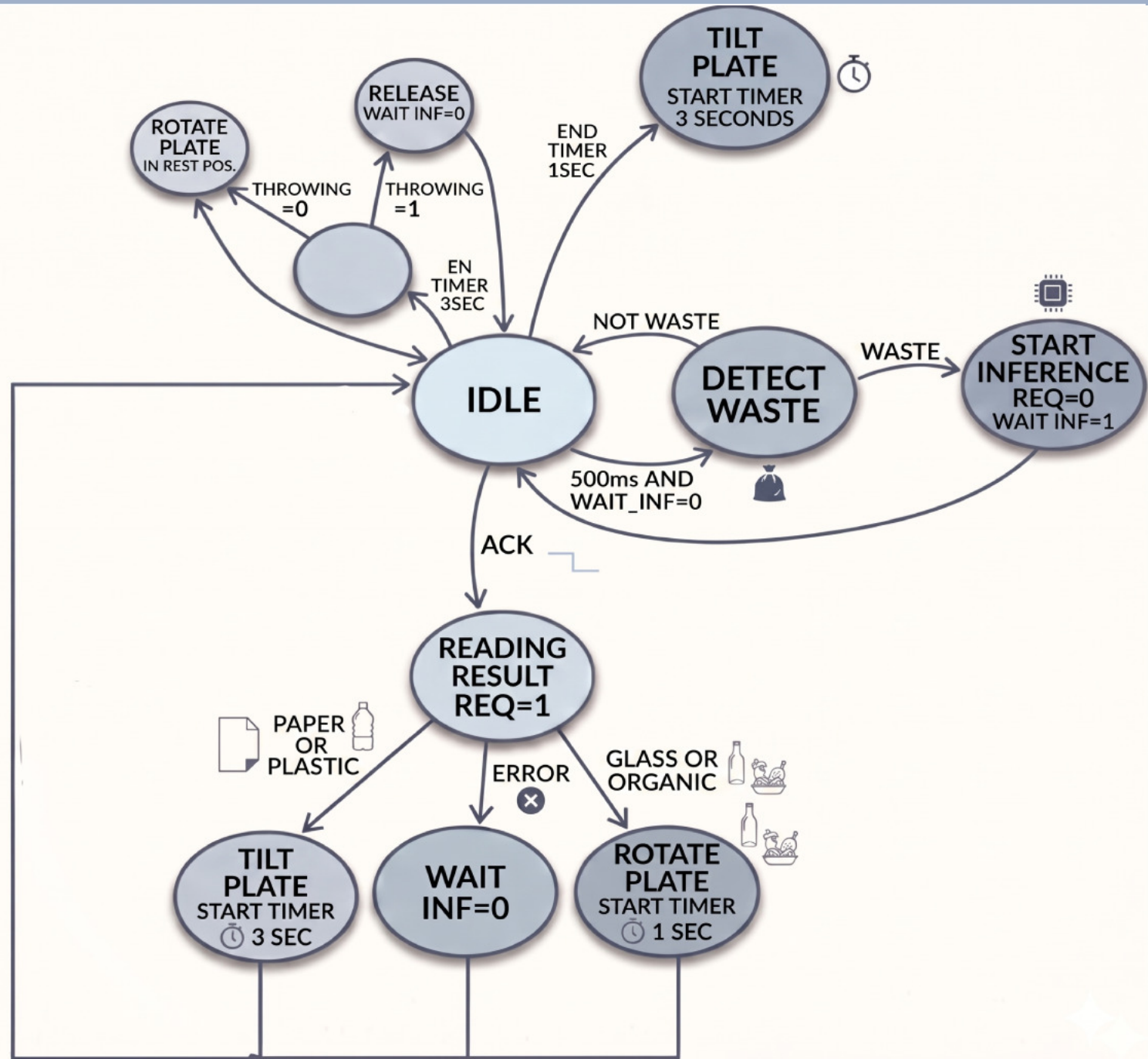
2. Returns a measurement that can be converted into distance

SG90 Servo Motor

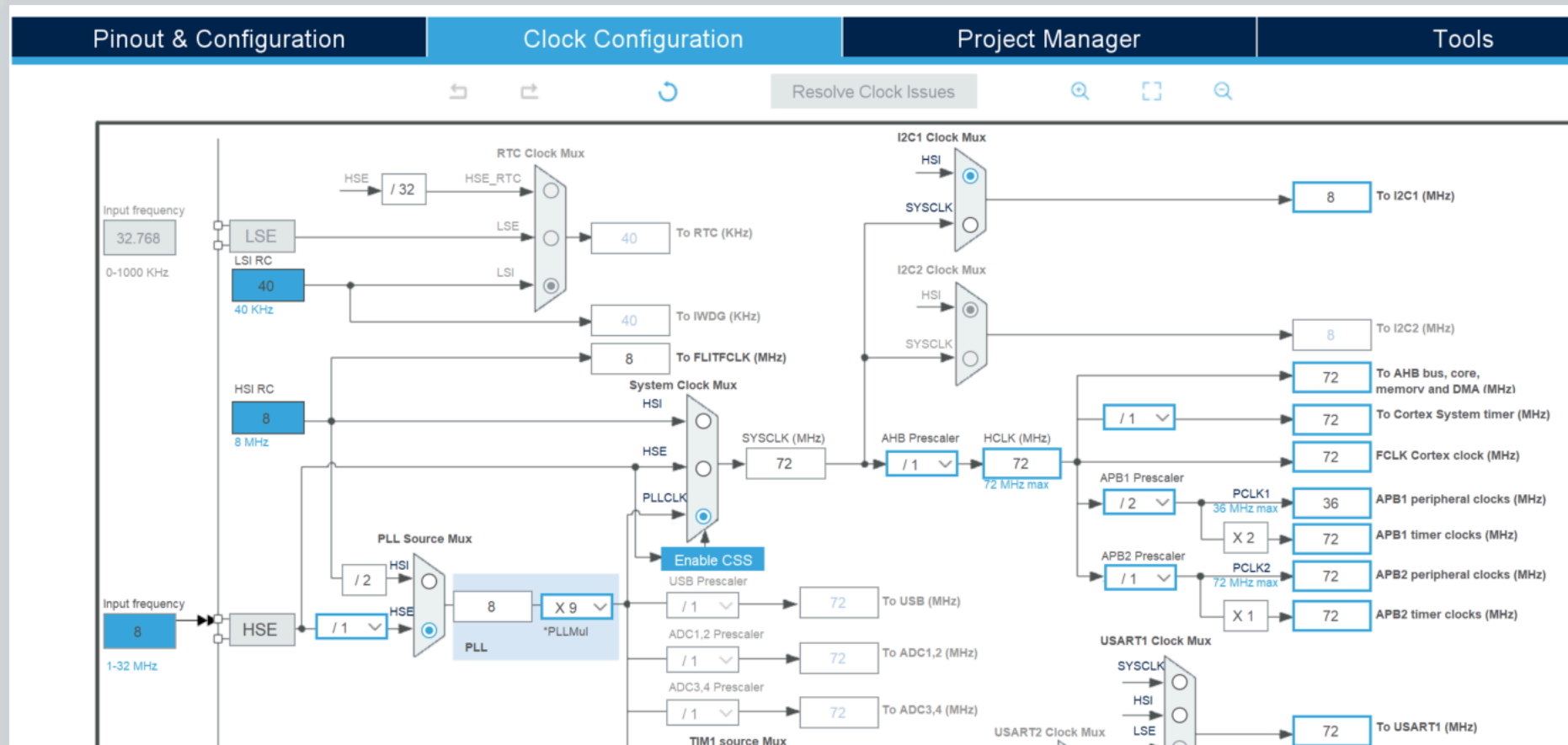
Simple movement/tilting of the platform thanks to the PWM signal



STATE MACHINE OF THE OPERATION CYCLE



OPERATING FREQUENCIES – SYSTEM CLOCK



- Clock configured at the maximum frequency (72 MHz)

OPERATING FREQUENCIES – TIMER I

- TIM1: PWM signal generation for servo motor control
- Channel 1: rotation of the support platform
- Channels 2 and 3: tilting of the support platform
- PSC: set to 71 in order to set TIM1 to a frequency of 1MHz

$$F_{tim} = \frac{F_{clk}}{PSC + 1} = \frac{72000000}{72} = 1MHz$$

The screenshot shows the STM32CubeMX configuration interface for the TIM1 peripheral. The top section contains dropdown menus for Slave Mode (Disable), Trigger Source (Disable), Clock Source (Internal Clock), and four channels (Channel1 to Channel4) set to PWM Generation CH1 through CH3, with Channel4 set to Disable. Below this is a 'Configuration' section with a 'Reset Configuration' button and four tabs: NVIC Settings, DMA Settings, GPIO Settings, and Parameter Settings (which is selected). Under the Parameter Settings tab, there is a search bar and a list of parameters to configure. The 'Counter Settings' section is expanded, showing the following values: Prescaler (PSC - 16 bits value) set to 71, Counter Mode set to Up, Counter Period (AutoReload Register) set to 19999, and Internal Clock Division (CKD) set to No Division.

Configuration	
Reset Configuration	
✓ NVIC Settings	✓ DMA Settings
✓ Parameter Settings	✓ GPIO Settings
Configure the below parameters :	
Search (Ctrl+F)	
Counter Settings	
Prescaler (PSC - 16 bits value)	71
Counter Mode	Up
Counter Period (AutoReload Register)	19999
Internal Clock Division (CKD)	No Division

OPERATING FREQUENCIES – TIMER 2

- Following a trigger on the TRIGGER pin, the HC-SR04 sensor sends a pulse on the ECHO pin.
The farther the object is, the longer the signal remains high
- TIM2: used to measure the duration of the ECHO signal generated by the ultrasonic proximity sensor
- PSC: set to 71 in order to set TIM2 to a frequency of 1MHz

$$F_{tim} = \frac{F_{clk}}{PSC + 1} = \frac{72000000}{72} = 1MHz$$

The screenshot shows the STM32CubeMX configuration window for Timer 2. The top section contains dropdown menus for Slave Mode (Disable), Trigger Source (Disable), Clock Source (Internal Clock), and four channels (Channel1 to Channel4, all set to Disable). Below this is a 'Configuration' section with a 'Reset Configuration' button and four tabs: 'Parameter Settings' (selected), 'User Constants', 'NVIC Settings', and 'DMA Settings'. Under the 'Parameter Settings' tab, a search bar is present. The 'Counter Settings' section is expanded, showing the following parameters: Prescaler (PSC - 16 bits value) set to 71, Counter Mode set to Up, Counter Period (AutoReload Register value) set to 4294967295, Internal Clock Division (CKD) set to No Division, and auto-reload preload set to Disable. The 'Trigger Output (TRGO) Parameters' section is also visible but not expanded.

Parameter	Value
Slave Mode	Disable
Trigger Source	Disable
Clock Source	Internal Clock
Channel1	Disable
Channel2	Disable
Channel3	Disable
Channel4	Disable

Configuration

Reset Configuration

Parameter Settings | User Constants | NVIC Settings | DMA Settings

Configure the below parameters :

Search (Ctrl+F)

Counter Settings

Prescaler (PSC - 16 bits value)	71
Counter Mode	Up
Counter Period (AutoReload Register value)	4294967295
Internal Clock Division (CKD)	No Division
auto-reload preload	Disable

Trigger Output (TRGO) Parameters

OPERATING FREQUENCIES – TIMER 3

- TIM3: used during the final phase of the Operation Cycle
 1. Unloading Wait: when the platform tilts, TIM3 is started to wait 3 seconds so that the object slides off before the system straightens the platform
 2. Stabilization: when the platform returns to the horizontal position it may be subject to oscillations. TIM3 is started again to block sensor measurements for another 3 seconds
- PSC: set to 9999 in order to set TIM3 to a frequency of 7200Hz. To count up to 3 seconds, it is necessary to count up to $7200 * 3 = 21600$
- $ARR = 21600 - 1 = 21599$ because counting starts from 0

The screenshot shows the STM32CubeMX configuration window for Timer 3. At the top, there are dropdown menus for 'Trigger Source' (set to 'Disable'), 'Clock Source' (set to 'Internal Clock'), and four channels (Channel1, Channel2, Channel3, Channel4), all set to 'Disable'. Below these is a 'Configuration' section with a 'Reset Configuration' button and four tabs: 'Parameter Settings' (selected), 'User Constants', 'NVIC Settings', and 'DMA Settings'. Under the 'Parameter Settings' tab, there is a search bar and a list of parameters to configure. The 'Counter Settings' section includes: Prescaler (PSC - 16 bits value) set to 9999, Counter Mode set to 'Up', Counter Period (AutoReload Regi...) set to 21599, Internal Clock Division (CKD) set to 'No Division', and auto-reload preload set to 'Disable'. The 'Trigger Output (TRGO) Parameters' section shows Master/Slave Mode (MSM bit) set to 'Disable (Trigger input effect not delayed)'.

Parameter	Value
Trigger Source	Disable
Clock Source	Internal Clock
Channel1	Disable
Channel2	Disable
Channel3	Disable
Channel4	Disable

Configuration

Reset Configuration

✓ Parameter Settings ✓ User Constants ✓ NVIC Settings ✓ DMA Settings

Configure the below parameters :

Search (Ctrl+F)

Counter Settings

Prescaler (PSC - 16 bits value)	9999
Counter Mode	Up
Counter Period (AutoReload Regi...)	21599
Internal Clock Division (CKD)	No Division
auto-reload preload	Disable

Trigger Output (TRGO) Parameters

Master/Slave Mode (MSM bit)	Disable (Trigger input effect not delayed)
-----------------------------	--

OPERATING FREQUENCIES – TIMER 4

- TIM4: used during the Disposal phase when the platform needs to rotate before tilting
- When the base starts rotating, TIM4 is started to wait 1 second
- PSC: set to 9999 in order to set TIM4 to a frequency of 7200Hz. To count up to 1 second, it is necessary to count up to 7200
- $ARR = 7200 - 1 = 7199$ because counting starts from 0

The screenshot shows the STM32CubeMX configuration window for Timer 4. The top section shows the basic configuration: Trigger Source is set to 'Disable', Clock Source is 'Internal Clock', and all four channels (Channel1 to Channel4) are set to 'Disable'. Below this is a 'Configuration' section with a 'Reset Configuration' button. A tabbed interface shows 'Parameter Settings' as the active tab, with other tabs for 'User Constants', 'NVIC Settings', and 'DMA Settings'. Under 'Parameter Settings', a search bar is present. The 'Counter Settings' section is expanded, showing: Prescaler (PSC - 16 bits value) set to 9999, Counter Mode set to 'Up', Counter Period (AutoReload Register) set to 7199, Internal Clock Division (CKD) set to 'No Division', and auto-reload preload set to 'Disable'. The 'Trigger Output (TRGO) Parameters' section is also expanded, showing Master/Slave Mode (MSM bit) set to 'Disable (Trigger input effect not delayed)'.

Trigger Source	Disable
Clock Source	Internal Clock
Channel1	Disable
Channel2	Disable
Channel3	Disable
Channel4	Disable

Configuration

Reset Configuration

Parameter Settings | User Constants | NVIC Settings | DMA Settings

Configure the below parameters :

Search (Ctrl+F)

Counter Settings

Prescaler (PSC - 16 bits value)	9999
Counter Mode	Up
Counter Period (AutoReload Register)	7199
Internal Clock Division (CKD)	No Division
auto-reload preload	Disable

Trigger Output (TRGO) Parameters

Master/Slave Mode (MSM bit)	Disable (Trigger input effect not delayed)
-----------------------------	--

GPIO CONFIGURATION – ECHOPIN

- ECHOPIN: configured in External Interrupt mode on both the Rising and Falling Edges:

1. The rising-edge interrupt occurs when the sensor generates the ECHO signal

2. The falling-edge interrupt occurs when the sensor detects the reflected signal

The interrupt handler stores the initial and final values of the TIM2 timer in order to calculate the pulse duration

The screenshot shows the STM32CubeMX Configuration window. On the left, the 'DMA' section is expanded, showing 'GPIO' selected. The 'Configuration' tab is active, and the 'Group By Peripherals' dropdown is set to 'GPIO'. The 'Search Signals' field is empty. The 'Show only Modified Pins' checkbox is checked. The table below lists the configured pins and their settings.

Pin	Signal	GPIO ou...	GPIO m...	GPIO P...	Maximu...	Fast Mode	User La...	Modified
PA9	n/a	n/a	External...	No pull-...	n/a	n/a	ECHOPIN	✓
PB10	n/a	Low	Output ...	No pull-...	Low	n/a	REQ	✓
PB11	n/a	n/a	Input m...	No pull-...	n/a	n/a	bit0	✓
PB12	n/a	n/a	Input m...	No pull-...	n/a	n/a	bit1	✓
PB13	n/a	n/a	Input m...	No pull-...	n/a	n/a	bit2	✓
PB14	n/a	n/a	External...	No pull-...	n/a	n/a	ACK	✓
PC9	n/a	Low	Output ...	No pull-...	Low	n/a	TRIGG...	✓

Below the table, the 'PA9 Configuration' section is expanded, showing the following settings:

- GPIO mode: External Interrupt Mode with Rising/Falling edge trigger detection
- GPIO Pull-up/Pull-down: No pull-up and no pull-down
- User Label: ECHOPIN

NOTE: the ultrasonic sensor generates the ECHO signal following a trigger from the STM32 on the TRIGGER PIN

GPIO CONFIGURATION – STM-RASPBERRY HANDSHAKE

- The REQ/ACK pins manage the handshake:

1. REQ: configured as Output. Since it is high at rest, when the measured distance indicates the presence of a waste object, the STM32 pulls this signal low to request inference from the Raspberry Pi
2. ACK: configured as an External Interrupt on the Falling Edge. When the Raspberry has finished inference, it switches the ACK signal from 1 to 0, generating an interrupt on the STM32

The screenshot shows the STM32CubeMX Configuration window. On the left, a sidebar lists various peripherals: DMA, GPIO (checked), IWDG, NVIC (checked), RCC (checked), SYS (checked), TSC, and WWDG. Below this, there are expandable sections for Analog, Timers, Connectivity, Multimedia, Computing, and Middleware and Software. The main area is titled 'Configuration' and shows a 'Group By Peripherals' dropdown set to 'GPIO'. Below this, there are tabs for various peripherals: GPIO (checked), I2C, RCC, SYS, TIM, USART, and NVIC. A 'Search Signals' box is present with the text 'Search (Ctrl+F)'. To the right of the search box is a checkbox labeled 'Show only Modified Pins'. Below these elements is a table of pins and their configurations. The table has columns: Pin, Signal, GPIO output, GPIO mode, GPIO pull-up/down, Maximum frequency, Fast mode, User Label, and Modified. The rows are: PA9, PB10, PB11, PB12, PB13, PB14, and PC9. PB14 is highlighted in blue. Below the table, there is a section titled 'PB14 Configuration :'. It contains three fields: 'GPIO mode' set to 'External Interrupt Mode with Falling edge trigger detection', 'GPIO Pull-up/Pull-down' set to 'No pull-up and no pull-down', and 'User Label' set to 'ACK'. A red arrow points from the 'PB14' row in the table to the 'PB14 Configuration' section.

Pin	Signal	GPIO output	GPIO mode	GPIO pull-up/down	Maximum frequency	Fast mode	User Label	Modified
PA9	n/a	n/a	External...	No pull-...	n/a	n/a	ECHOPIN	✓
PB10	n/a	Low	Output ...	No pull-...	Low	n/a	REQ	✓
PB11	n/a	n/a	Input m...	No pull-...	n/a	n/a	bit0	✓
PB12	n/a	n/a	Input m...	No pull-...	n/a	n/a	bit1	✓
PB13	n/a	n/a	Input m...	No pull-...	n/a	n/a	bit2	✓
PB14	n/a	n/a	External...	No pull-...	n/a	n/a	ACK	✓
PC9	n/a	Low	Output ...	No pull-...	Low	n/a	TRIGG...	✓

PB14 Configuration :

GPIO mode: External Interrupt Mode with Falling edge trigger detection

GPIO Pull-up/Pull-down: No pull-up and no pull-down

User Label: ACK

GPIO CONFIGURATION – PARALLEL DATA BUS

- The bit0/bit1/bit2 pins constitute the actual data transmission channel between the STM and Raspberry
- Configured as Input
- The Raspberry encodes the inference result using 3 bits (Paper, Plastic/Metal, Glass, Organic, Error)
- After receiving the ACK signal, the STM32 reads the inference result from the parallel bus

Configuration

Group By Peripherals

GPIO I2C RCC SYS TIM USART NVIC

Search Signals

Search (Ctrl+F) ☐ Show only Modified Pins

Pin	Signal	GPIO ou...	GPIO m...	GPIO P...	Maximu...	Fast Mode	User La...	Modified
PA9	n/a	n/a	External...	No pull-...	n/a	n/a	ECHOPIN	✓
PB10	n/a	Low	Output ...	No pull-...	Low	n/a	REQ	✓
PB11	n/a	n/a	Input m...	No pull-...	n/a	n/a	bit0	✓
PB12	n/a	n/a	Input m...	No pull-...	n/a	n/a	bit1	✓
PB13	n/a	n/a	Input m...	No pull-...	n/a	n/a	bit2	✓
PB14	n/a	n/a	External...	No pull-...	n/a	n/a	ACK	✓
PC9	n/a	Low	Output ...	No pull-...	Low	n/a	TRIGG...	✓

PB14 Configuration :

GPIO mode: External Interrupt Mode with Falling edge trigger detection

GPIO Pull-up/Pull-down: No pull-up and no pull-down

User Label: ACK

NVIC CONFIGURATION

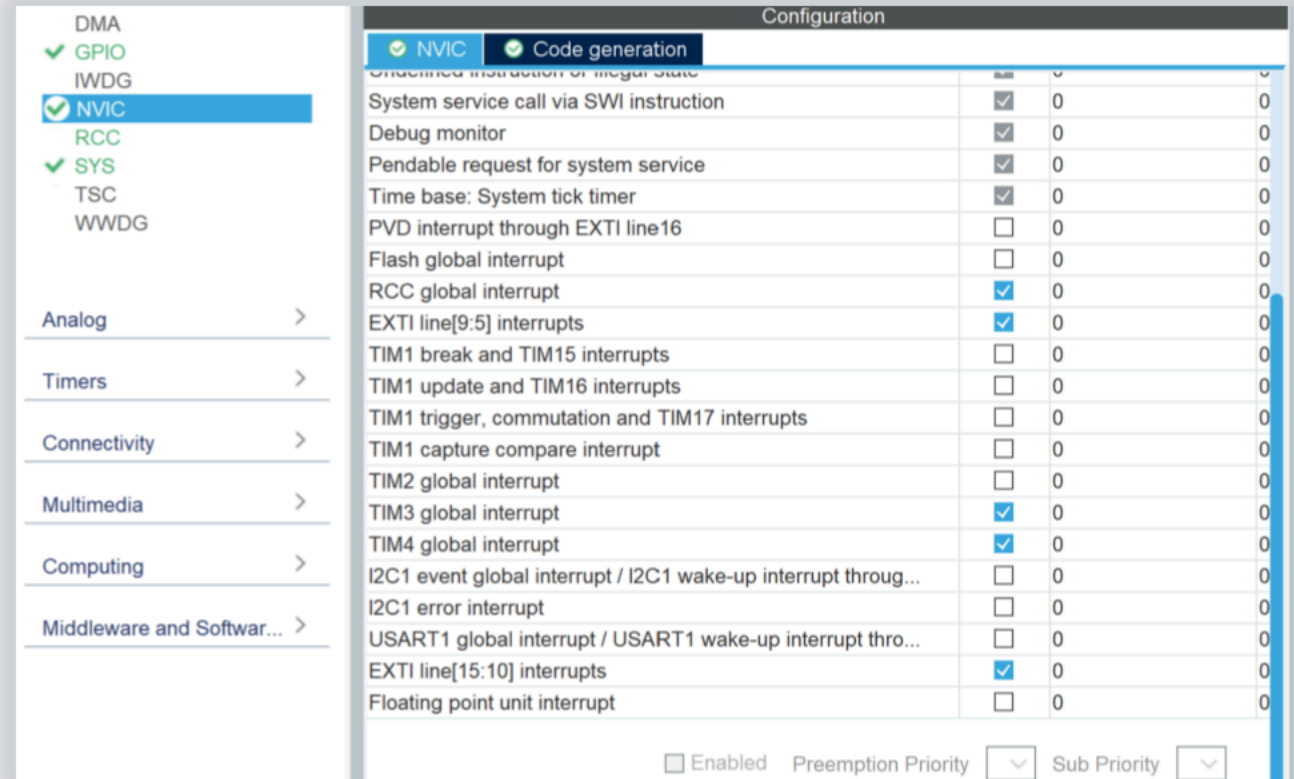
- Enabling 3 fundamental interrupt lines:

1. EXTI line[9:5]: for interrupts on ECHOPIN

2. EXTI line[15:10]: for interrupts on ACK

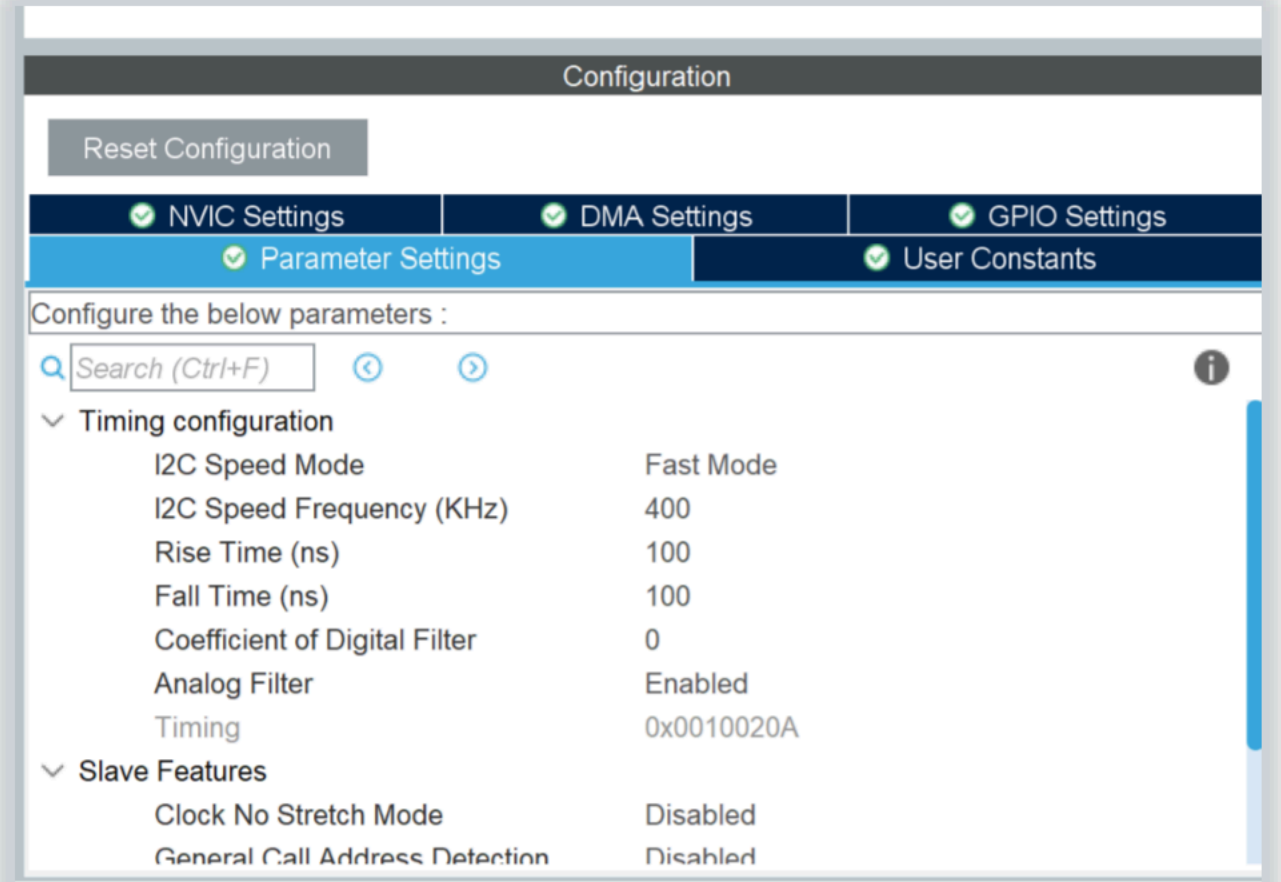
3. TIM3 Global Interrupt: so that TIM3 generates an interrupt when the 3-second period expires

4. TIM4 Global Interrupt: so that TIM4 generates an interrupt when the 1-second period expires



I2C CONFIGURATION

- I2C configured in Fast Mode
(required for the Displays to operate)



Configuration

Reset Configuration

✓ NVIC Settings ✓ DMA Settings ✓ GPIO Settings

✓ Parameter Settings ✓ User Constants

Configure the below parameters :

Search (Ctrl+F) ⏪ ⏩ ⓘ

▼ Timing configuration

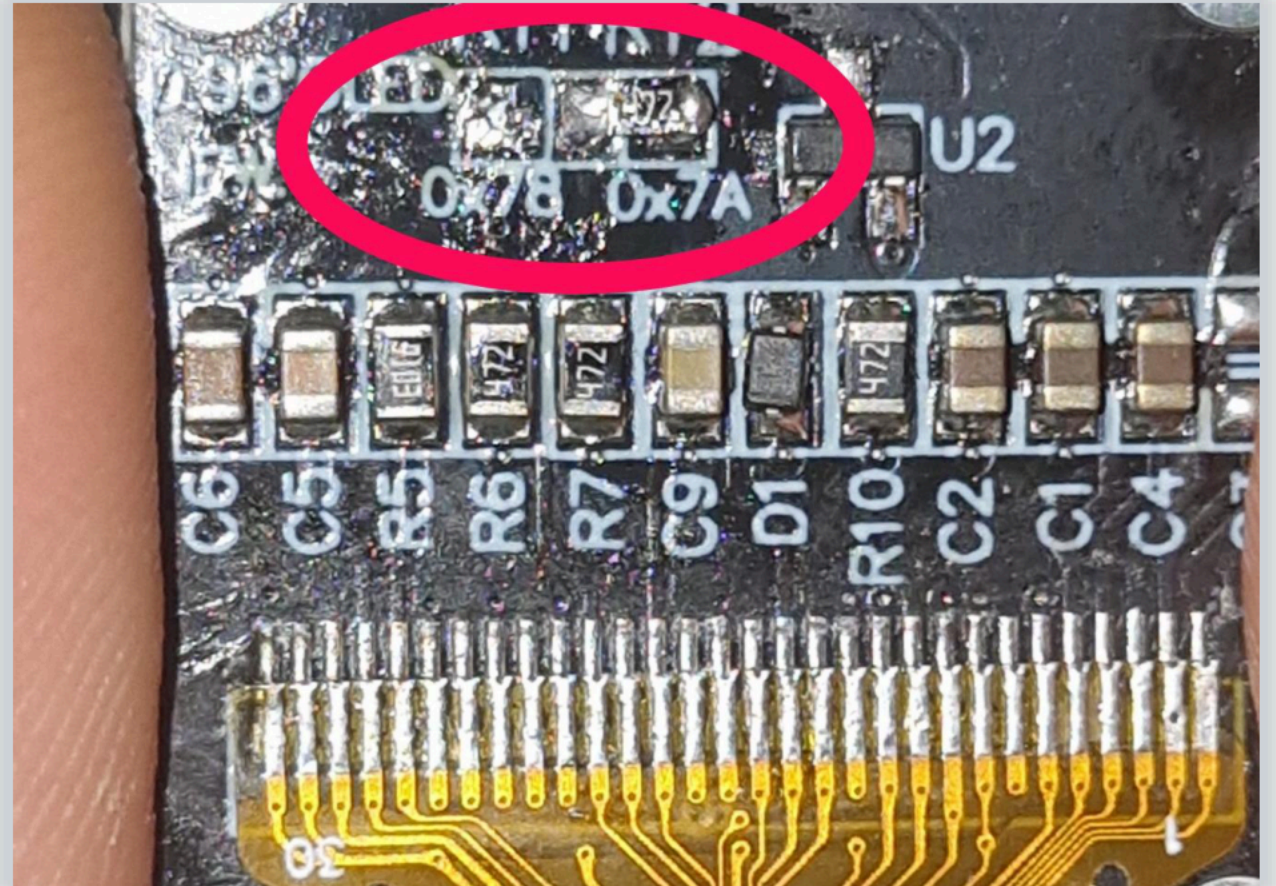
I2C Speed Mode	Fast Mode
I2C Speed Frequency (KHz)	400
Rise Time (ns)	100
Fall Time (ns)	100
Coefficient of Digital Filter	0
Analog Filter	Enabled
Timing	0x0010020A

▼ Slave Features

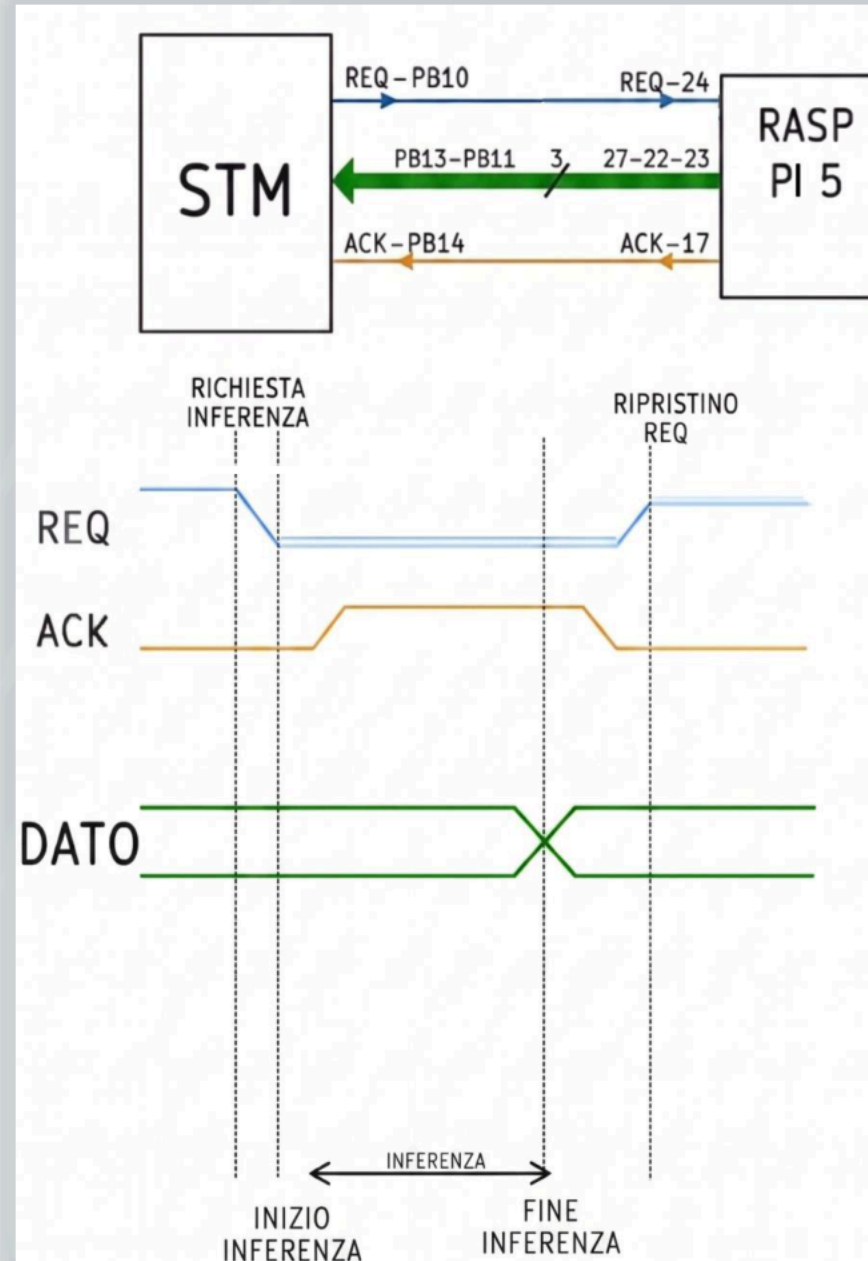
Clock No Stretch Mode	Disabled
General Call Address Detection	Disabled

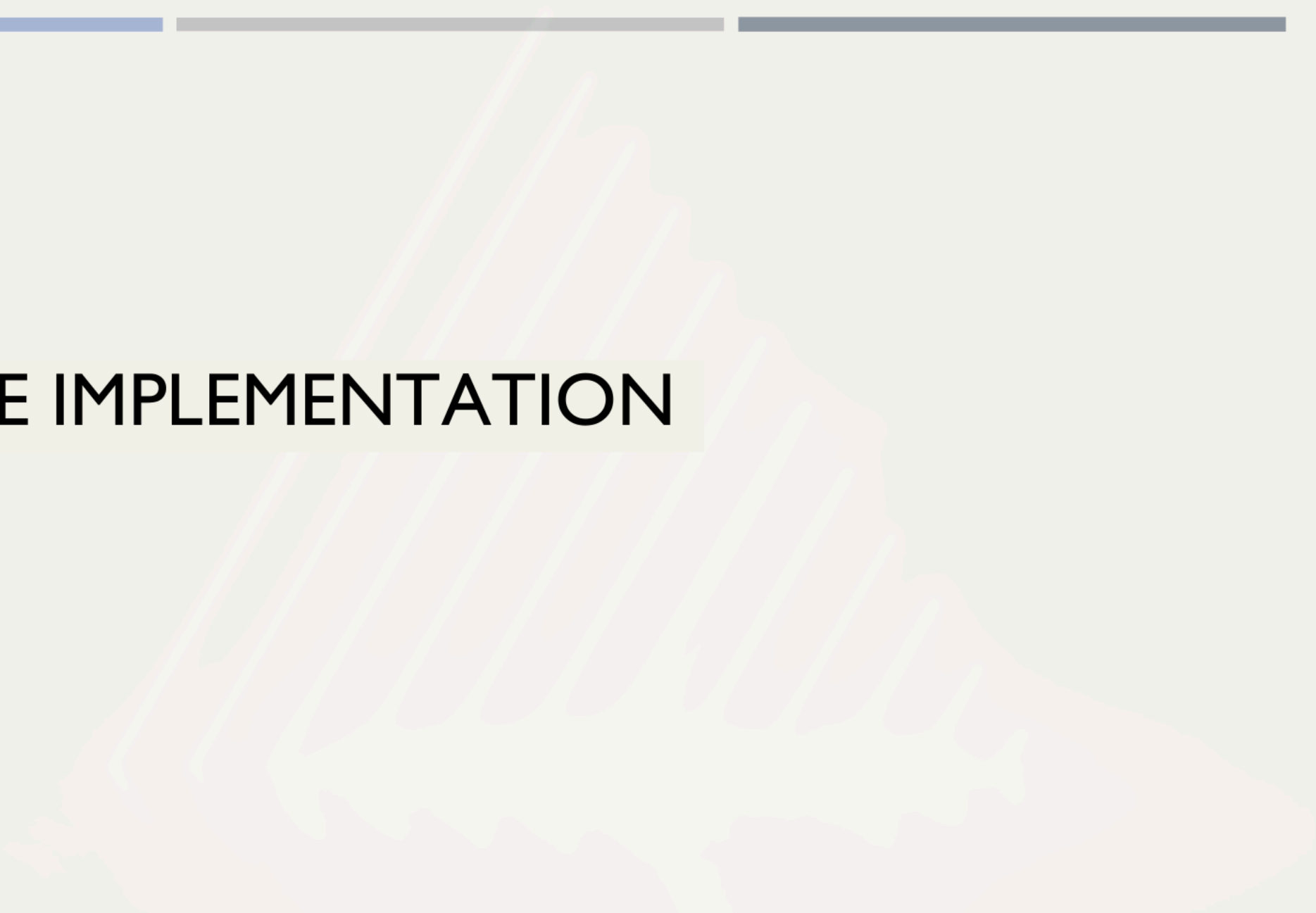
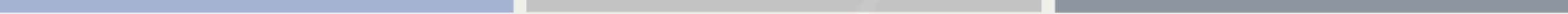
DISPLAY MODIFICATION

- The system uses two identical OLED Displays
- By default, these devices have the same hardware address, making it impossible to distinguish the devices
- To solve the problem, a physical modification was made to the back of the printed circuit board of one of the two displays
- The resistor used to configure the address was moved from address 0x78 to address 0x7A



TIMING DIAGRAM STM- RASPBERRY





SOFTWARE IMPLEMENTATION

PRELIMINARY OPERATIONS



```
// Imposta il pin REQ alto (stato di riposo)
HAL_GPIO_WritePin(PARA_PORT, REQ_Pin, GPIO_PIN_SET);
```

```
// Azzerata e avvia TIM2 per la lettura del segnale ECHO
__HAL_TIM_SET_COUNTER(&htim2, 0);
HAL_TIM_Base_Start(&htim2);
```

```
// Avvia i segnali PWM su TIM1 per il controllo dei 3 servomotori
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_3);
```

```
// Inizializzazione e Scrittura sui display OLED
SSD1306_Init(1);
SSD1306_Init(2);
```

```
SSD1306_Clear(1);
SSD1306_GotoXY(0, 0);
SSD1306_Puts("THROW", &Font_11x18, 1);
SSD1306_GotoXY(0, 30);
SSD1306_Puts("SOMETHING", &Font_11x18, 1);
SSD1306_UpdateScreen(1);
```

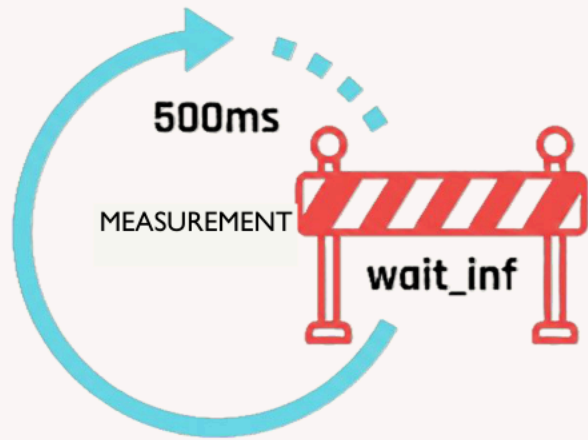
```
SSD1306_Clear(2);
SSD1306_GotoXY(0, 30);
SSD1306_Puts("CONCESTINO", &Font_11x18, 1);
SSD1306_UpdateScreen(2);
```

```
// Imposta i servomotori del piatto in posizione di riposo
__HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 500);
__HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, 1389);
__HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, 1389);

// Reset dei flag UPDATE su TIM3 e TIM4 per prevenire bug di avvio
__HAL_TIM_CLEAR_FLAG(&htim3, TIM_FLAG_UPDATE);
__HAL_TIM_CLEAR_FLAG(&htim4, TIM_FLAG_UPDATE);

/* USER CODE END 2 */
```

STARTING THE MEASUREMENT



- Every 500 ms, the STM sends a 10 ms trigger pulse
- If the system is performing inference (`wait_inf=true`), the system does not allow any other measurements

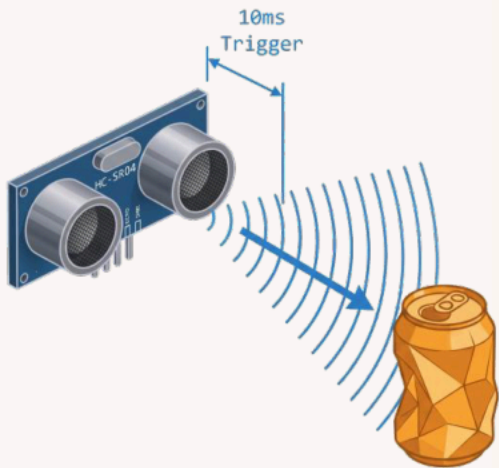
```
/* USER CODE BEGIN WHILE */
while (1) {
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */

  /* A MENO CHE NON SI STIA ATTENDENDO LA FINE DI UNA INFERENZA EFFETTUIAMO UNA NUOVA MISURA */
  if(!wait_inf){
    avvio_misura();
    HAL_Delay(500);
  }
}
/* USER CODE END 3 */
```

```
void avvio_misura(){
  // STIMOLO DEL SENSORE CON IMPULSO DI 10ms
  HAL_GPIO_WritePin(TRIGGERPORT,TRIGGERPIN_Pin, GPIO_PIN_SET);
  HAL_Delay(10);
  HAL_GPIO_WritePin(TRIGGERPORT,TRIGGERPIN_Pin, GPIO_PIN_RESET);
}
```

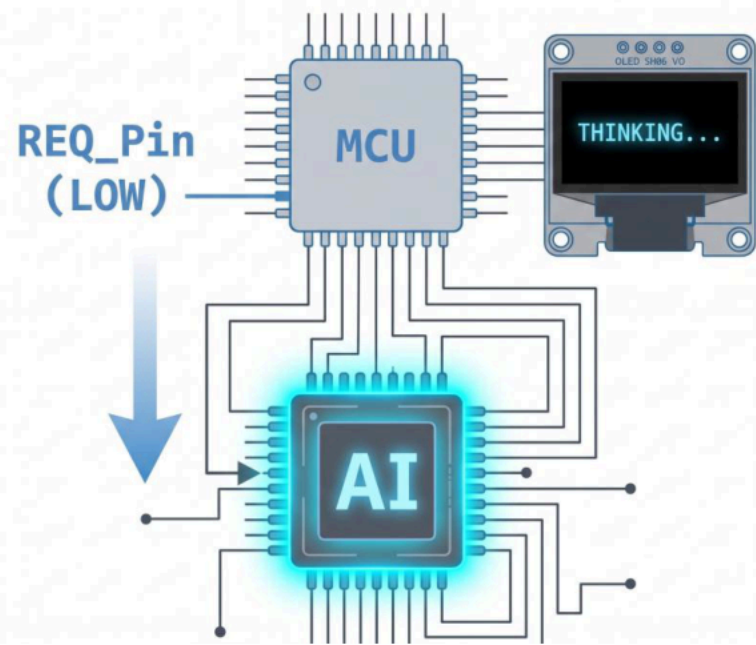
OBJECT DETECTION



```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {  
  
    /* Controllo se l'eccezione e' dovuta da una variazione del segnale ECHO generato dal sensore */  
  
    if (GPIO_Pin == ECHOPIN_Pin && wait_inf==0) {  
  
        /* Controllo se l'interruzione e' sul fronte di salita o discesa*/  
  
        if (HAL_GPIO_ReadPin(ECHOPORT, ECHOPIN_Pin) == GPIO_PIN_SET) {  
  
            /* Se e' sul fronte di salita salvo il valore di conteggio iniziale di TIM2 */  
            __HAL_TIM_SET_COUNTER(&htim2, 0);  
            echo_start = __HAL_TIM_GET_COUNTER (&htim2);  
        } else {  
  
            /* Altrimenti salvo il valore di conteggio finale di TIM2 */  
            echo_stop = __HAL_TIM_GET_COUNTER (&htim2);  
  
            /* Tramite questi due valori salvati e' possibile determinare la distanza  
            dell'oggetto mediante la seguente formula: */  
            distance = (echo_stop-echo_start)* 0.034/2;  
  
            if (distance >= 30) distance = 30; /* Per correggere l'overflow mettiamo una saturazione */  
  
            /* Se la distanza misurata e' minore della soglia riteniamo che sia stato  
            effettivamente rilevato un oggetto */  
            if(distance<soglia){  
  
                /* Avviamo l'inferenza e stoppiamo successive misure tramite la variabile wait_inf */  
                wait_inf=1;  
                inferenza();  
            }  
        }  
    }  
}
```

- This Callback is executed every time an EXTI-type interrupt occurs
- We check whether we need to handle the interrupt generated by the ultrasonic sensor
- On the rising edge, I save the initial TIM2 count value
- On the falling edge, I save the final TIM2 count value
- Using these values, we calculate the distance of the object

INFERENCE REQUEST



- STM pulls the REQ signal low
- Since wait_inf has been set high, measurements are interrupted

```
void inferenza(){  
    /* Per richiedere un'inferenza dobbiamo abbassare il segnale REQ */  
    HAL_GPIO_WritePin(PARA_PORT,REQ_Pin,GPIO_PIN_RESET);  
  
    /* Stampa su Display */  
    SSD1306_Clear(1);  
    SSD1306_GotoXY(0, 0);  
    SSD1306_Puts("THINKING...", &Font_11x18, 1);  
    SSD1306_UpdateScreen(1);  
}
```

END OF INFERENCE

- When the ACK signal goes low, inference is finished
- The interrupt is always handled in the callback used for the sensor (void HAL_GPIO_EXTI_Callback)
- We read the inference result
- Finally, we can call the function to dispose of the waste (throw)

```
else if(GPIO_Pin == ACK_Pin){  
  
    /* Gestione dell'interruzione EXTI dovuta al fronte di discesa  
    del segnale di ACK */  
  
    /* Leggiamo il risultato dell'inferenza */  
    res=0;  
    res |= HAL_GPIO_ReadPin(PARA_PORT, bit0_Pin) << 0; // bit 0  
    res |= HAL_GPIO_ReadPin(PARA_PORT, bit1_Pin) << 1; // bit 1  
    res |= HAL_GPIO_ReadPin(PARA_PORT, bit2_Pin) << 2; // bit 2  
  
    /* Settiamo alto il segnale REQ per future richieste */  
    HAL_GPIO_WritePin(PARA_PORT, REQ_Pin, GPIO_PIN_SET);  
  
    /* Dopo aver letto il risultato possiamo gettare il rifiuto nel corretto cestino */  
    throw();  
}  
}
```

THROW FUNCTION

```
void throw(){
    char stampa[16]={'\0'};

    switch(res){

        case CARTA: {
            sprintf(stampa, "CARTA");
            /* Per gettare la carta che si trova nel verso della posizione di
            riposo, incliniamo direttamente il piatto */
            __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 500);
            __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, 944);
            __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, 1833);

            /* Avvio il TIM3 per riportare il piatto in posizione di equilibrio */
            HAL_TIM_Base_Start_IT(&htim3);
            break;
        }

        case PLASTICA: {
            sprintf(stampa, "PLAST/MET");
            /* Per gettare la plastica/metallo che si trova nel verso della posizione di
            riposo, incliniamo direttamente il piatto */
            __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 500);
            __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, 1833);
            __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, 944);

            /* Avvio il TIM3 per riportare il piatto in posizione di equilibrio */
            HAL_TIM_Base_Start_IT(&htim3);
            break;
        }
    }
}
```

```
case VETRO: {
    sprintf(stampa, "VETRO");
    /* Per gettare il vetro e' necessario ruotare il piatto,
    avviamo poi TIM4 per aspettare 1s prima di inclinare il piatto */
    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 1500);
    vetroorg=1;

    /* Avvio il TIM4 */
    HAL_TIM_Base_Start_IT(&htim4);
    break;
}

case ORGANICO: {
    sprintf(stampa, "ORGANICO");
    /* Per gettare l'organico e' necessario ruotare il piatto,
    avviamo poi TIM4 per aspettare 1s prima di inclinare il piatto */
    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 1500);
    vetroorg=2;

    /* Avvio il TIM4 */
    HAL_TIM_Base_Start_IT(&htim4);
    break;
}

default: {
    /* Se arriva una codifica diversa dalle precedenti si assume un errore nell'inferenza */
    sprintf(stampa, "ERROR");
    wait_inf=0;
    break;
}
}
```

TIMER CALLBACK – TIM4

- As with GPIO, the same Callback is always executed for Timers (void HAL_TIM_PeriodElapsedCallback). Therefore, we need to check which Timer generated the interrupt and handle it
- When the platform rotates, TIM4 is started to wait 1s before tilting the platform
- The global variable vetroorg determines whether the material is glass/organic so that the platform tilts in the correct direction (it is set by the throw function)

```
else if (htim->Instance == TIM4) {  
    /* TIM4: Timer di 1 secondo */  
    /* Fermo il Timer */  
    HAL_TIM_Base_Stop_IT(&htim4);  
  
    /* Inclino il piatto */  
    if(vetroorg==1){  
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, 944);  
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, 1833);  
    }else if(vetroorg==2){  
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, 1833);  
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, 944);  
    }  
    vetroorg=0;  
  
    /* Avvio il TIM3 per riportare il piatto in posizione di equilibrio */  
    HAL_TIM_Base_Start_IT(&htim3);  
}
```

TIMER CALLBACK – TIM3

- TIM3 1st Interrupt: used to wait 3 seconds before returning the platform to its rest position after the platform has tilted

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {  
  
    /* TIM3: Timer di 3 secondi */  
  
    if (htim->Instance == TIM3) {  
        /* Se throwing=0 --> e' la prima volta che scatta TIM3 */  
        if (!throwing) {  
            /* Fermo il Timer */  
            HAL_TIM_Base_Stop_IT(&htim3);  
  
            /* Riporto il piatto in posizione di equilibrio */  
            __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 500);  
            __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, 1389);  
            __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, 1389);  
  
            /* Settiamo a 1 la var globale throwing in modo che  
            la prossima volta che TIM3 genera un'interruzione  
            l'esecuzione della Callback va nell'altro ramo */  
            throwing = 1;  
  
            /* Avvio di nuovo TIM3 (altri 3s) in modo da  
            garantire la stabilizzazione del piatto */  
            HAL_TIM_Base_Start_IT(&htim3);  
        }  
    }  
}
```

- TIM3 2nd Interrupt: used to wait 3 seconds before restarting measurements, so that the platform can stabilize without oscillations causing detections

```
} else {  
    /* Allo scadere dei 3s viene eseguita di nuovo la callback.  
    Adesso throwing=1 --> piatto stabilizzato  
    --> riattivo la rilevazione */  
  
    /* Fermo il Timer */  
    HAL_TIM_Base_Stop_IT(&htim3);  
  
    /* Reset della var globale throwing */  
    throwing = 0;  
  
    /* Sblocco misurazioni */  
    wait_inf=0;  
  
    /* Scrittura su Display */  
    SSD1306_Clear(1);  
    SSD1306_GotoXY(0, 0);  
    SSD1306_Puts("THROW", &Font_11x18, 1);  
    SSD1306_GotoXY(0, 30);  
    SSD1306_Puts("SOMETHING", &Font_11x18, 1);  
    SSD1306_UpdateScreen(1);  
}
```

PYTHON SCRIPT FOR RASPBERRY – I

```
1  #Librerie necessarie:
2  #per la webcam:
3  import cv2
4  from PIL import Image
5  #per il modello:
6  import torch
7  import torch.nn.functional as F
8  from torchvision import transforms
9  #per i gpio:
10 import RPi.GPIO as GPIO
11 #Altro:
12 from time import sleep
13
14 #Mnemonici Pin:
15 PIN_ACK = 17
16 PIN_BIT2 = 27
17 PIN_BIT1 = 22
18 PIN_BIT0 = 23
19 PIN_REQ = 24
20
21 #Variabili:
22 temp0 = 0
23 temp1 = 0
24 temp2 = 0
25
26 GPIO.setmode(GPIO.BCM)
27 #Configurazione dei PIN
28 GPIO.setup(PIN_ACK, GPIO.OUT)
29 GPIO.setup(PIN_BIT0, GPIO.OUT)
30 GPIO.setup(PIN_BIT1, GPIO.OUT)
31 GPIO.setup(PIN_BIT2, GPIO.OUT)
32 GPIO.setup(PIN_REQ, GPIO.IN)
33 GPIO.output(PIN_ACK, GPIO.HIGH)
```

```
35 #Preparazione del modello:
36 PERCORSO_MODELLO = './APC/modello_rifiuti_v3.pt'
37 CLASSI = ['glass', 'metal', 'organic', 'paper', 'plastic']
38 PERCORSO_SALVATAGGIO = 'res.jpg'
39 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
40 print(f"Caricamento del modello su {device}...")
41 model = torch.jit.load(PERCORSO_MODELLO, map_location=device)
42 model.eval()
43 data_transforms = transforms.Compose([
44     transforms.Resize(256),
45     transforms.CenterCrop(224),
46     transforms.ToTensor(),
47     transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
48 ])
```

PYTHON SCRIPT FOR RASPBERRY – II

```
50 #CICLO INFINITO, in cui si attendono le richieste del raspberry:
51 while True:
52     #Se il segnale di REQ si abbassa significa che è arrivata una richiesta.
53     if not GPIO.input(PIN_REQ):
54         print("req ricevuta")
55
56         #Ricevuta la richiesta rialziamo il segnale ACK
57         GPIO.output(PIN_ACK, GPIO.HIGH)
58
59         #ricevuta la richiesta attendiamo 2.5s per dare il tempo di rimuovere la mano dal piatto
60         sleep(2.5)
61         #avvio webcam
62         cap = cv2.VideoCapture(0)
63         if not cap.isOpened():
64             print("Errore: Impossibile accedere alla webcam.")
65             exit()
66         #dopo aver avviato la webcam, attendiamo mezzo secondo affinché possa mettere a fuoco.
67         sleep(0.5)
68         # Cattura di un singolo fotogramma dalla webcam
69         ret, frame = cap.read()
```

```
#Gestione della dimensione e ritaglio dell'immagine in modo
#che venga dato in inferenza solo ciò sul piatto.
#troviamo il centro dell'immagine:
h, w, _ = frame.shape
side = min(h, w)
# Coordinate del centro
cx, cy = w // 2, h // 2
#Coordinate del crop
x1 = cx - 290 // 2
y1 = cy - 350 // 2
x2 = cx + 370 // 2
y2 = cy + 300 // 2
#Ritaglio immagine
frame = frame[y1:y2, x1:x2]

# Conversione formato colori: OpenCV usa il formato colori BGR, ma PyTorch si aspetta RGB.
frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
img_pil = Image.fromarray(frame_rgb)
img_tensor = data_transforms(img_pil).unsqueeze(0).to(device)
```

PYTHON SCRIPT FOR RASPBERRY – III

```
#Predizione:
with torch.no_grad():
    outputs = model(img_tensor)
    # Calcolo della probabilità con Softmax
    probabilita = F.softmax(outputs, dim=1)[0]
    #Preleviamo la probabilità più alta e il suo indice.
    valore_max, indice_predetto = torch.max(probabilita, 0)

# Estraiamo il nome della classe e la percentuale
classe_predetta = CLASSI[indice_predetto.item()]
percentuale = valore_max.item() * 100

if(classe_predetta=='plastic' or classe_predetta=='metal'):
    classe_predetta='plast/met'
```

#CODIFICHIAMO LA CLASSE PREDETTA

match classe_predetta:

case 'paper':

temp0=1

temp1=0

temp2=0

case 'plast/met':

temp0=0

temp1=1

temp2=0

case 'glass':

temp0=1

temp1=1

temp2=0

case 'organic':

temp0=0

temp1=0

temp2=1

case _:

temp0=0

temp1=0

temp2=0

#Stampa debug

print(str(temp2)+"-"+str(temp1)+"-"+str(temp0))

print(classe_predetta)

#Scrittura della codifica sul BUS

GPIO.output(PIN_BIT0, temp0)

GPIO.output(PIN_BIT1, temp1)

GPIO.output(PIN_BIT2, temp2)

#il dato è pronto e si abbassa il segnale di ACK

GPIO.output(PIN_ACK, GPIO.LOW)

print("ho abbassato ack")

#Salviamo l'immagine catturata e data in inferenza come debug e poter capire
#cosa il modello vede.

salvato = cv2.imwrite(PERCORSO_SALVATAGGIO, frame)

if salvato:

print(f"Frame salvato in: {PERCORSO_SALVATAGGIO}")

else:

print("Errore: impossibile salvare il frame.")

#Chiusura webcam e finestra

cap.release()

cv2.destroyAllWindows()



... HOW WOULD AN AI TOOL
HAVE SOLVED OUR PROBLEM?

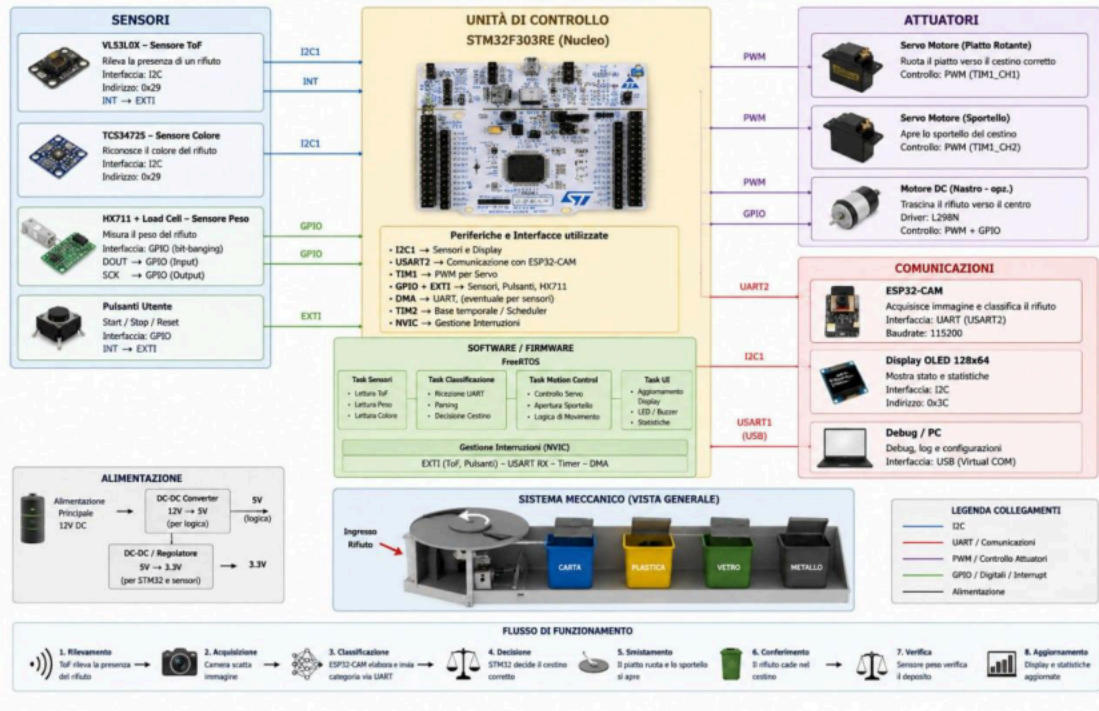
THE PROMPT USED

« We are a group of 4 Master's students in Computer Engineering who have to develop a project for the APC (Computer Architecture and Design) exam. The project guidelines require us to use at least one STM32F3 board and to highlight our ability to use this board, its various communication interfaces, the various protocols and devices present on the board. We need to know how to correctly use interrupts, interact with sensors and actuators, and so on. Our idea is to build a smart waste bin, i.e. a system that detects the presence of a waste object and throws it into the correct category, for example by means of a moving platform. Could you help us design it, for example by explaining: Which components to use and which protocols to use to interact with them. The architectural scheme and how to connect the various devices to each other. How to recognize the waste object. How to configure the board and the code to use. Thank you. »

PROPOSED ARCHITECTURES COMPARED

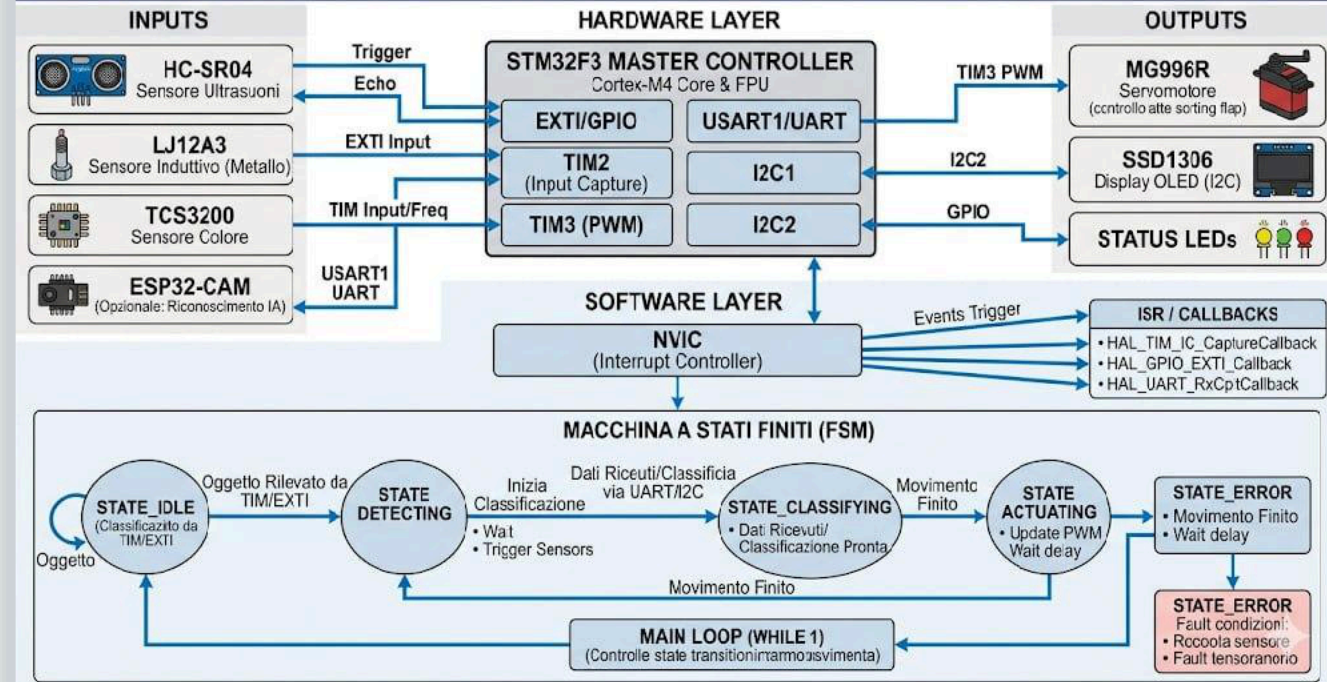
ChatGPT

ARCHITETTURA DEL SISTEMA - SMART BIN



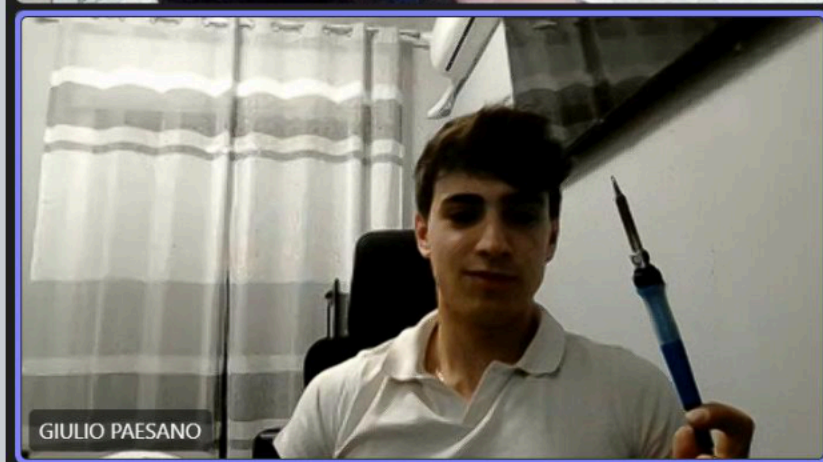
Google Gemini

ARCHITETTURA SISTEMA EMBEDDED: CESTINO SMART STM32F3



https://chatgpt.com/s/t_6a1093fbc3888191b1e70256c6e43f62

<https://gemini.google.com/share/eb32695e5296>



THANK YOU FOR
YOUR
ATTENTION